

Brushing (Chapter 14)

This section contains additional information that is not present in the book.

Customizing the brush

You can customize the appearance of the overlay, selection area and selection handles using brush methods and CSS. When a brush is created, it generates the following SVG:

```
<g class="brush" ... >
  <rect class="overlay" pointer-events="all" cursor="crosshair"/>
  <rect class="selection" fill="#777" fill-opacity="0.3" stroke="#fff" cursor="move"/>
  <rect class="handle handle--n" cursor="ns-resize"/>
  <!-- ... 7 more handles ... -->
</g>
```

Size attributes (x, y, height, width), not shown in the code above, are also generated dynamically for each rectangle. You can programmatically change any of these attributes.

The extent determines the size of the overlay. Use CSS to change colors, cursors, strokes, and opacities. For example, this stylesheet applies a yellow fill and black stroke to the selection rectangle, and shows the corner handles:

```
.selection {                      /* The selection rectangle */
  fill: none;
  stroke: black;
  fill-opacity: 0.75;
}
.handle {                         /* All handles */
  opacity: 0.75;
}
.handle:hover {
  stroke: white;
  fill: red;
}
.handle--ne, .handle--sw, .handle--nw, .handle--se { /* Corner handles */
  fill: black;
  opacity: 1;
}
```

Figure 1, at left, shows all CSS classes you can use to configure brush elements. To set the handle sizes, however, it's best to use the `handleSize()` method (`handleSize(0)` disables resizing):

```
brush.handleSize(10);
```

The effect of these changes is shown in *Figure 1*, at right. The right border handle ('handle-e') appears in red as the user resizes the selection area.

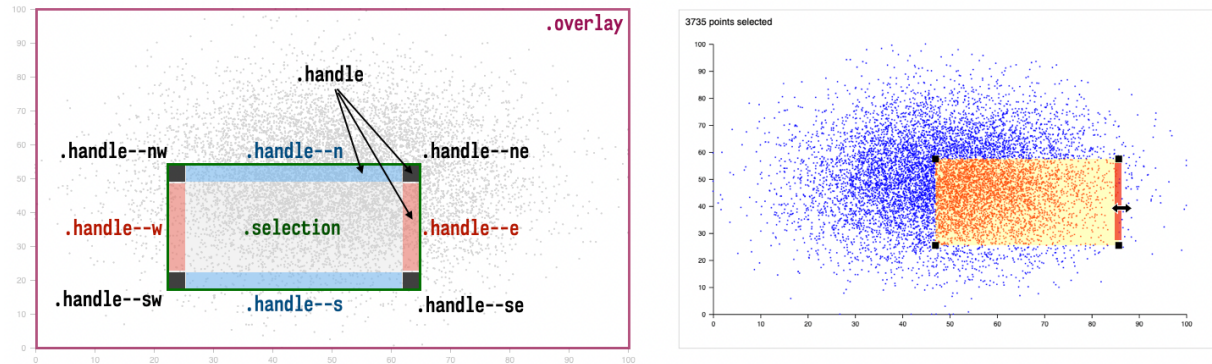


Figure 1 – Customizing a brush with CSS and modes. Code: Brush/9-styles.html.

Modes, activated when a brush is moved or resized, also allow for customization. The following snippet changes the brush color while moving it:

```
.on("brush", e => {
  if(e.mode === "drag") {
    d3.selectAll(".selection").style("fill", "orange");
  }
  /* ... */
});
```

See Brush/9-styles.html, to try out the changes described in this section.

Brushing over canvas

Brushing over a canvas uses the same mechanism used for brushing in SVG, since the brush is implemented in SVG. You just need to stack both your canvas and your SVG in the same position. This can be done with CSS:

```
canvas, svg { position: absolute; }
```

The Canvas should be added to the page *before* the SVG, so that the brush can capture user events.

Let's see a Canvas version of the code used in the book (see Brush/8-xy-canvas.html). The code for the chart is mostly the same as Brush/4-xy-data.html, except for the rendering, which plots the data points in a Canvas:

```
const canvas = d3.select("canvas").attr("width", dim.width).attr("height", dim.height);
const ctx = canvas.node().getContext("2d");
```

```

ctx.fillStyle = "blue";
ctx.beginPath();
ctx.rect(dim.margin, dim.margin, dim.width-dim.margin*2, dim.height-dim.margin*2);
ctx.clip();
data.forEach(d => drawPoint(xScale(d[0]), yScale(d[1]), 1));

```

The `drawPoint()` function, called for each data point, renders a dot on the Canvas:

```

function drawPoint(cx, cy, r) {
  ctx.beginPath();
  ctx.arc(cx, cy, r, 0, 2 * Math.PI);
  ctx.fill();
}

```

The 'brush' event is similar to the SVG example, but uses Canvas to highlight the selected points:

```

brush.on("brush", function(evt, dat) {
  if (!isValid(evt)) return;
  d3.select(this)
    .datum(data.filter(d => inBrush(evt.selection, [xScale(d[0]), yScale(d[1])]))))
  d3.select("#count").text(dat.length);
  ctx.clearRect(0, 0, dim.width, dim.height);
  data.forEach(d => {
    ctx.fillStyle = dat.includes(d) ? "red" : "blue";
    drawPoint(xScale(d[0]), yScale(d[1]), 1);
  });
});

```

No changes are necessary in the 'end' event handler, but `updateView()` must redraw the Canvas when called. The chart should work exactly as the SVG version:

```

function updateView() {
  d3.select(".x-axis").transition().call(xAxis);
  d3.select(".y-axis").transition().call(yAxis);
  ctx.clearRect(0, 0, dim.width, dim.height);
  ctx.fillStyle = "blue";
  container.datum().forEach(d => drawPoint(xScale(d[0]), yScale(d[1]), 1));
  container.call(brush.clear);
}

```

A simple filter was enough to select the points here. Redrawing a full canvas, however, may affect performance. Techniques such as using `requestAnimationFrame()` and quadtree algorithms can improve it.